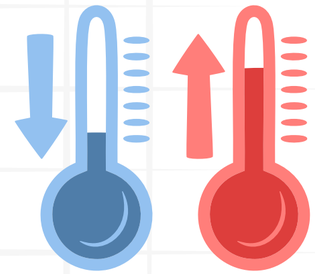
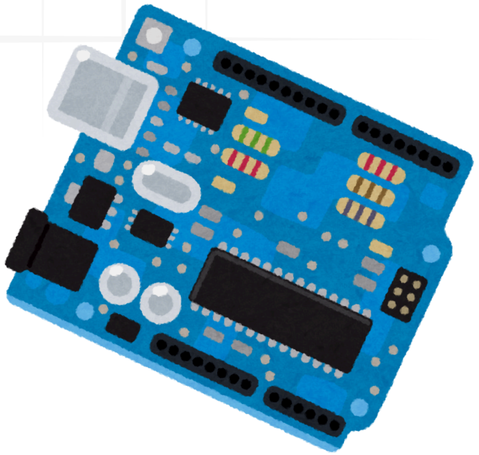
PDS – Semester Project – Fall 2025

Beverage Cooling Curve Recorder





Group 2 : Aisha Khalid
Khadim Hussain

Introduction

- The project used Arduino Uno with proper sensors to measure temperature of a beverage over time.
- This data is then used to train a polynomial regression model
- The model is then used to predict how many minutes beverage will take to reach drinkable temperature 40°C



Aim:

To predict the cooling time of beverages using temperature data.

Methodology

The project followed following steps:

Step 1 : Arduino Circuit Setup

Step 2 : Arduino Script

Step 3 : Data Collection

Step 4 : Data Processing

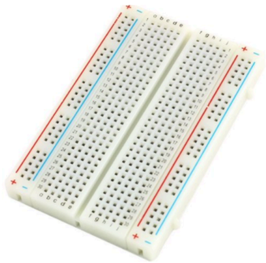
Step 5 : Training Polynomial Regression Model

Step 1 : Arduino Circuit Setup

Components:



Arduino Uno



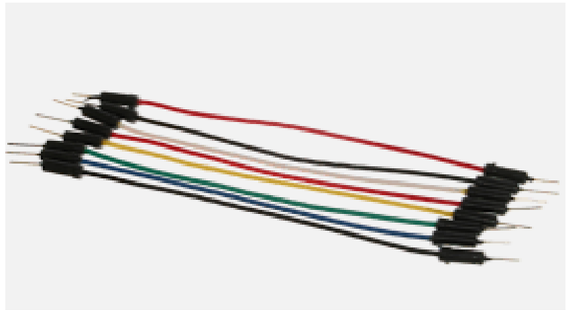
Bread Board



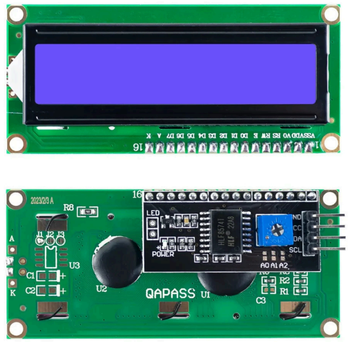
DS18B20 Temperature Sensor



10 k Ohm Resistor



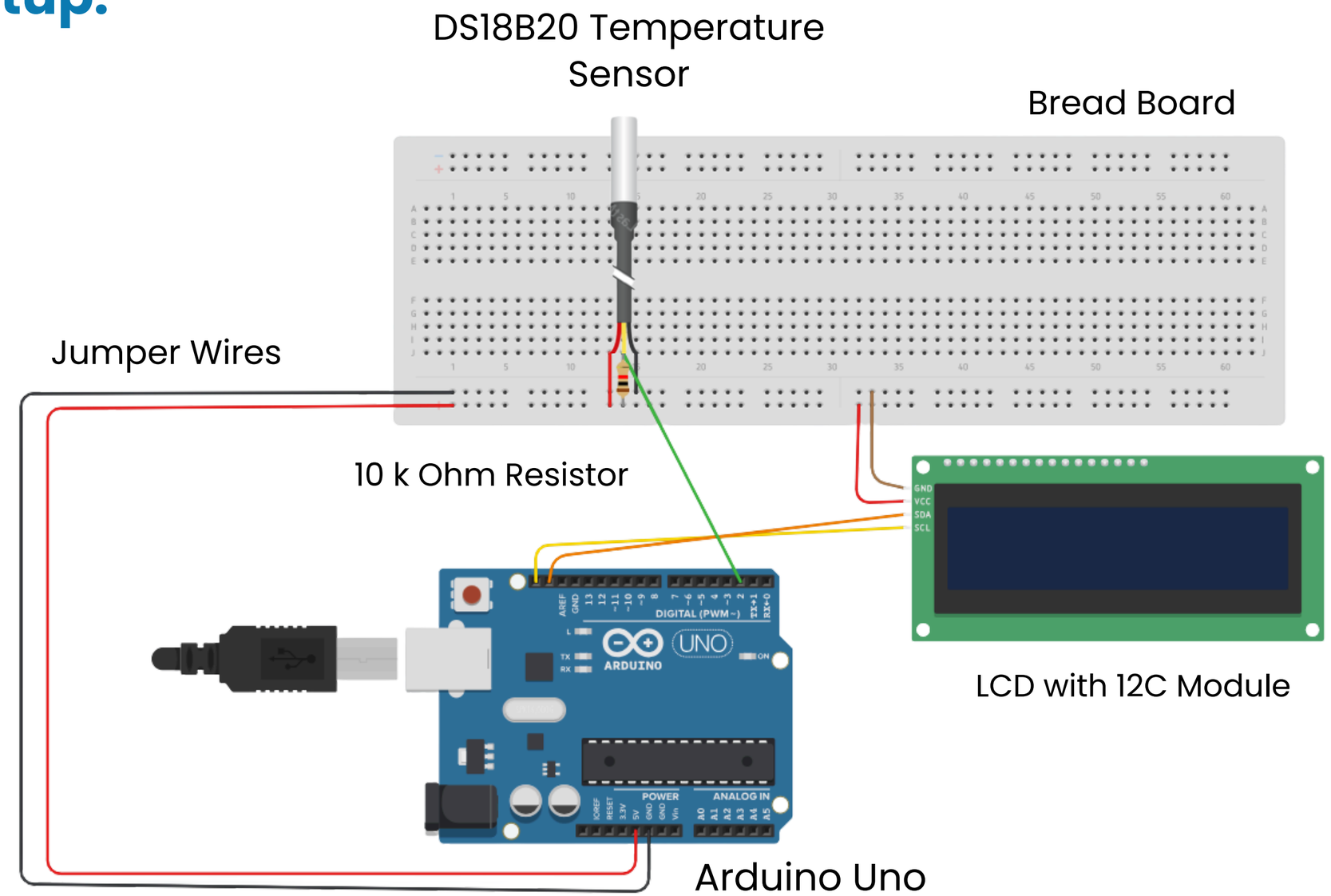
Jumper Wires



LCD with I2C Module

Step 1 : Arduino Circuit Setup

Circuit Setup:



Step 2 : Arduino Script

Overview:

- The Arduino script reads temperature data from a DS18B20 temperature sensor logs it every 10 second
- It displays the temperature readings on an LCD screen.
- It takes readings for 5 trails
- The reading starts logging when temperature increases a threshold of 55°C
- It end the current trail when temperature reaches a threshold of 36°C
- It also outputs the readings to the Serial Monitor for each trial to track the cooling process over time.

Step 2 : Arduino Script

Libraries Used:

- **OneWire:** Manages communication between the Arduino and the DS18B20 sensor using the OneWire protocol.
- **DallasTemperature:** Provides easy functions to interact with the DS18B20 sensor, including reading temperatures.
- **Wire:** Used to communicate with the LCD screen over I2C for displaying the temperature readings.
- **LiquidCrystal_I2C:** Controls the LCD screen to show the temperature readings in a user-friendly format.

Step 2 : Arduino Script

Script:

Project_Arduino_Sketch.ino

```ino
#include <OneWire.h>
#include <DallasTemperature.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Creating LCD object
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Data wire from Temp Sensor is plugged into digital pin 2
#define ONE_WIRE_BUS 2

// Thresholds
float start_threshold = 55.0;
float stop_threshold = 36.0;

// Setup a oneWire instance
OneWire oneWire(ONE_WIRE_BUS);

// Passing oneWire reference to DallasTemperature library
DallasTemperature sensors(&oneWire);

// Creating Variables
int trial_number = 1;  // Start at trial 1
int total_trials = 5;  // Total number of trials
bool logging_started = false;  // Data logging
unsigned long start_time = 0;  // Variable to track trial start time

void setup(void)
{
  sensors.begin();
  lcd.begin(16, 2);  // Initialize LCD
  lcd.backlight();   // Turn on backlight
  Serial.begin(9600);  // Start serial communication for user input

  Serial.println("Starting 5 trials...");
}

void loop(void)
{
  // Check if any trail is left
  if (trial_number <= total_trials) {
    // Read temperature data
    sensors.requestTemperatures();
    float temperatureC = sensors.getTempCByIndex(0);
```

Step 2 : Arduino Script

Script:

```
    // Display the temperature and trail number on the LCD
    lcd.setCursor(0, 0);
    lcd.print("Trial: ");
    lcd.print(trial_number);
    lcd.setCursor(0, 1);
    lcd.print("Temp: ");
    lcd.print(temperatureC);
    lcd.print((char)223);
    lcd.print("C");

    // Start logging when temperature crosses the start threshold
    if (temperatureC >= start_threshold && !logging_started) {
      start_time = millis();
      logging_started = true;
      Serial.print("Trial ");
      Serial.print(trial_number);
      Serial.println(" started. Logging temperature data...");
    }

    if (logging_started) {
      // Send temperature data to Serial Monitor for logging in Python
      unsigned long elapsed_time = (millis() - start_time) / 1000;  // Time in seconds
      Serial.print("Trial ");
      Serial.print(trial_number);
      Serial.print(", Time: ");
      Serial.print(elapsed_time);
      Serial.print("s, Temperature: ");
      Serial.println(temperatureC);

      // Stop logging when temperature reaches drinkable threshold
      if (temperatureC <= stop_threshold) {
        Serial.print("Trial ");
        Serial.print(trial_number);
        Serial.println(" completed. Drinkable temperature reached!");
        logging_started = false;  // Stop logging
        trial_number++;  // Move to the next trial
      }
    }

    // Wait for 10 seconds before reading again
    delay(10000);
  } else {
    // End the trials after completing the required number of trials
    Serial.println("All trials completed.");
    lcd.clear();
    lcd.print("Trials Completed");
    delay(2000);

  }
}
```

Step 3 : Data Collection

Overview:

The Python script reads real-time temperature data from the Arduino via a serial connection. It collects the temperature and time data for each trial, logs it into a CSV file, and also displays the data in the Serial Monitor for monitoring.

Key Functionality:

- **Trial Identification:** The script identifies the trial number based on the incoming data and associates it with the temperature and time readings.
- **CSV Logging:** For each trial, the script stores the data (trial number, elapsed time, and temperature) in a CSV file for further analysis.
- **Real-time Display:** It continuously prints the data to the console for live

Step 3 : Data Collection

Code:

```python
import serial
import time
import csv


ser = serial.Serial('COM3', 9600)

# Opening a CSV file to store data
filename = 'temperature_data.csv'
with open(filename, mode='w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(["Trial Number", "Time (s)", "Temperature (°C)"])  # headers

    # Collect data until the process is stopped
    while True:
        line = ser.readline().decode('utf-8').strip()  # Read a line from the serial port

        # Check for trial number and temperature data in the line
        if "Trial" in line:
            trial_number = line.split(' ')[1]  # Extract trial number

        if "Temperature" in line:
            # Extract time and temperature values
            time_data = line.split(",")[1].split("s")[0].strip()  # Elapsed time in seconds
            temp_data = line.split("Temperature: ")[1].strip()  # Temperature value

            # Log the data into the CSV file
            writer.writerow([trial_number, time_data, temp_data])

            # Print the data to the console for monitoring
            print(f"Trial {trial_number}, Time: {time_data}s, Temperature: {temp_data}°C")

        time.sleep(10)  # Wait 10 seconds before reading next data
```

Step 4 : Data Processing

Main Steps:

- Removing null readings
- Bringing in proper formatting for visualization

Temprature_Trails.xls

	A	B	C	D	E	F
1	Time (s)	Temperature (°C) - Trial 1	Temperature (°C) - Trial 2	Temperature (°C) - Trial 3	Temperature (°C) - Trial 4	Temperature (°C) - Trial 5
2	0	62.88	67.88	66.89	63.89	63.19
3	10	62.71	67.68	66.70	63.71	63.06
4	20	62.54	67.48	66.51	63.54	63

- Using Matplotlib to visualize cooling curve of each trail

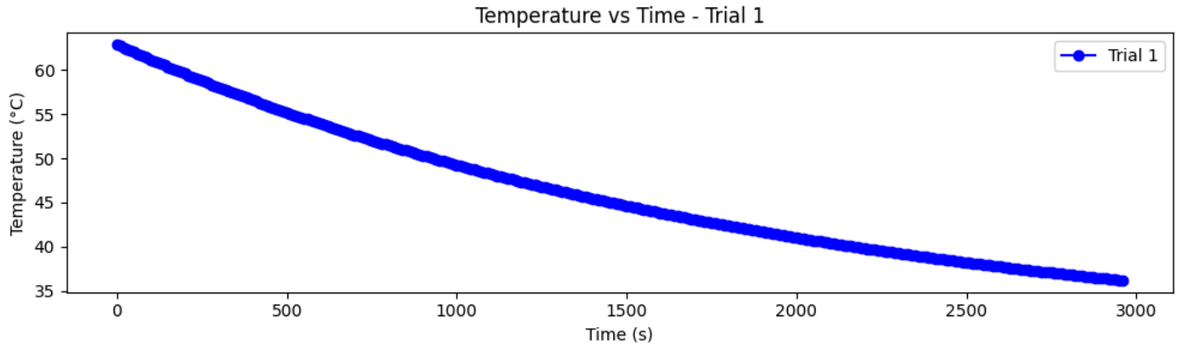
Choosing Trial 5 to be used for regression model training

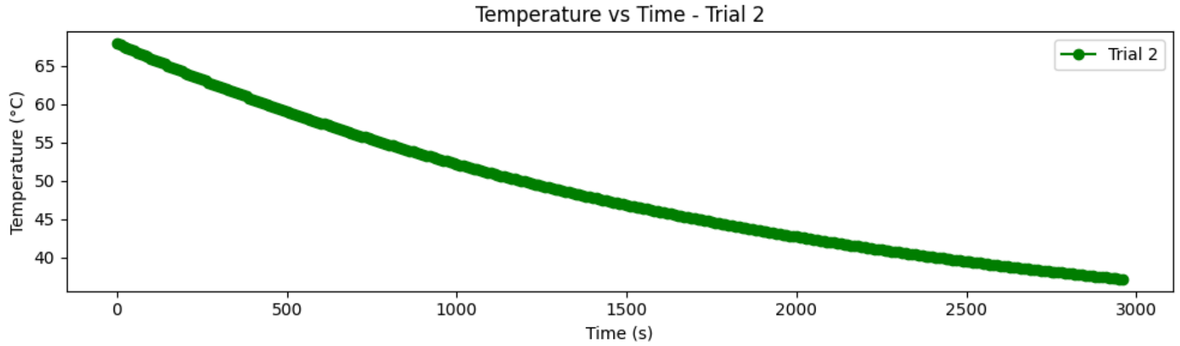
	A	B
1	Time	Temperature
2	0	63.19
3	10	63.06
4	20	63
5	30	62.81
6	40	62.63
7	50	62.5

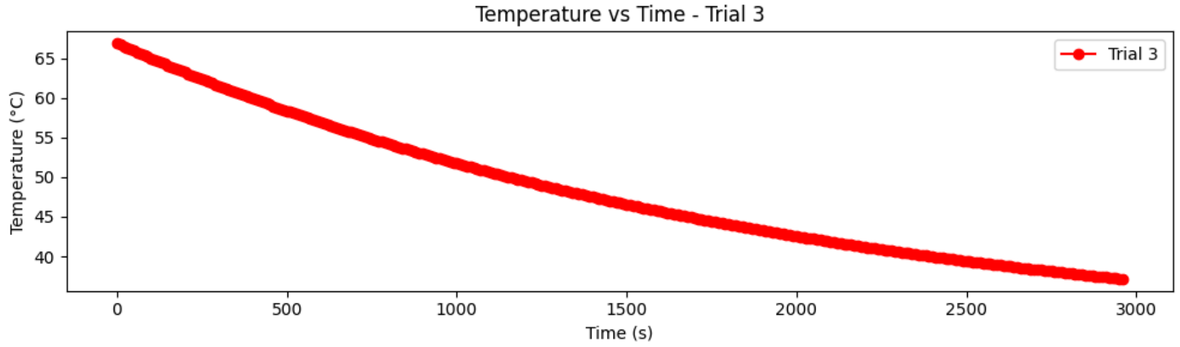
Regression_Trail.xls

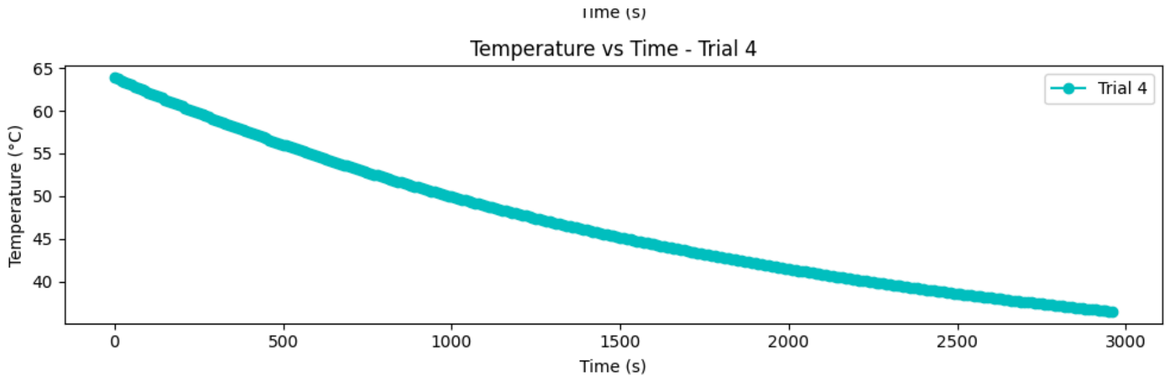
Step 4 : Data Processing

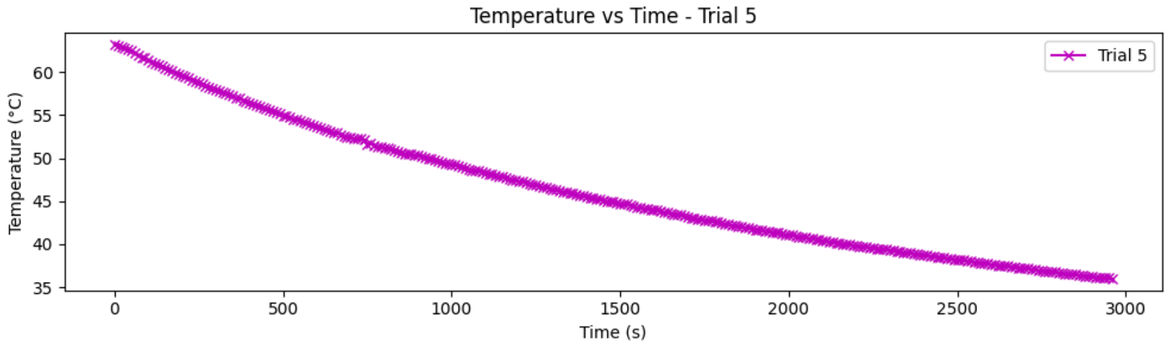
Visualization of Each Trial:











Step 5 : Training Polynomial Regression Model

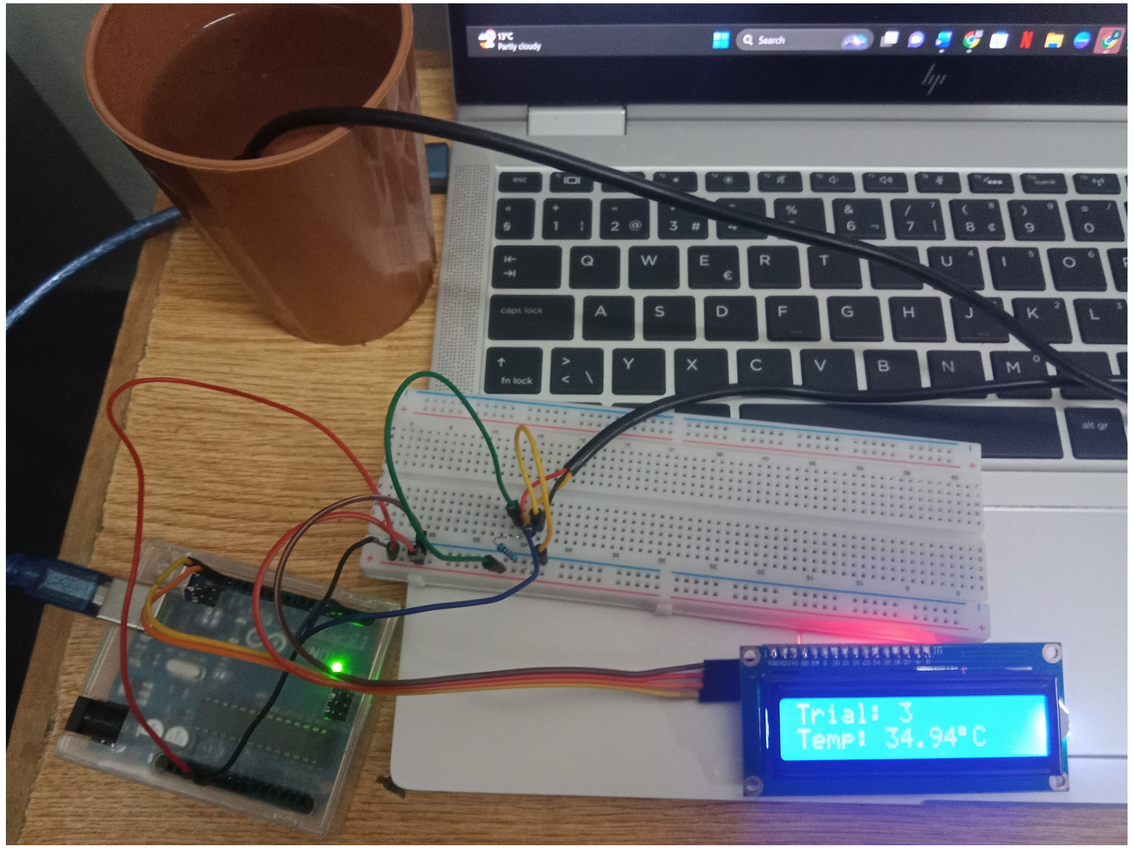
Overview:

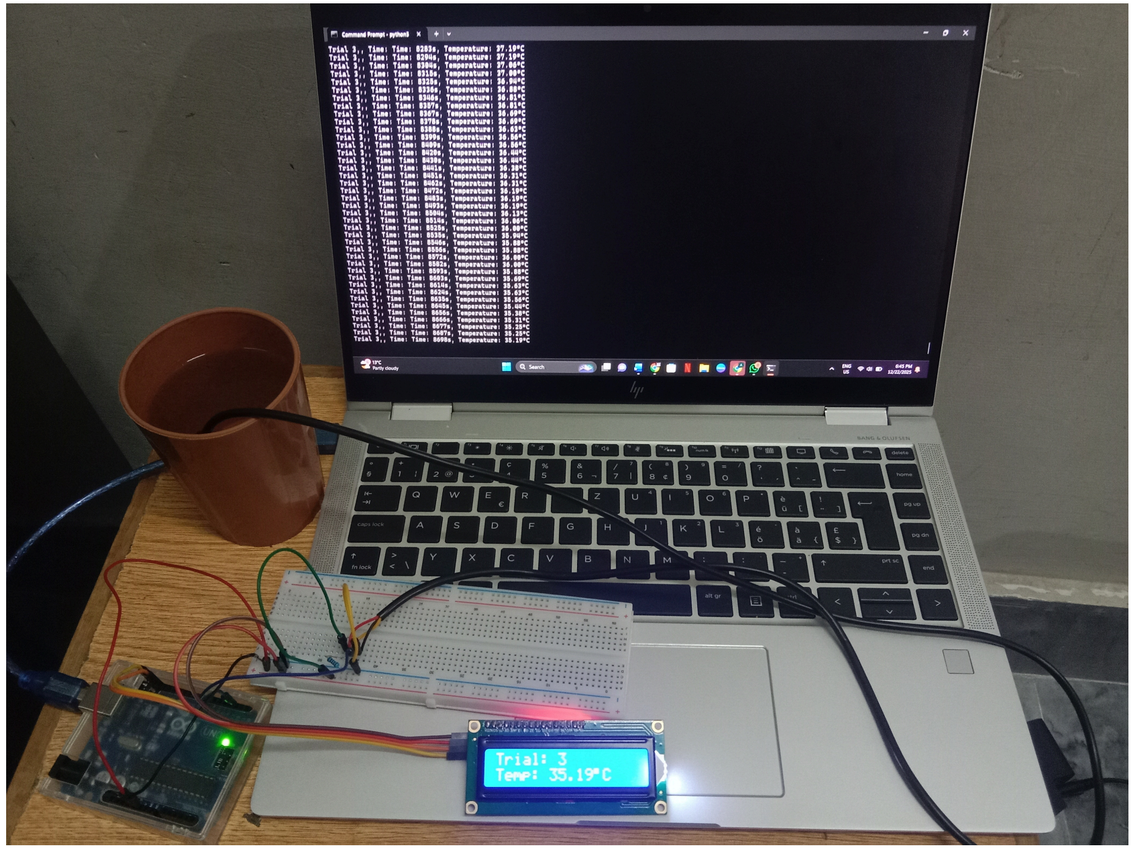
- In this step, we use the collected temperature and time data to train a Polynomial Regression model that captures the cooling curve of the beverage.
- The goal is to fit a polynomial model to the data and predict how long it will take for a beverage to cool to a target temperature.

Model Choice:

Polynomial Regression is chosen because it can fit non-linear relationships, such as the exponential-like cooling curve of beverages.

Snapshots:





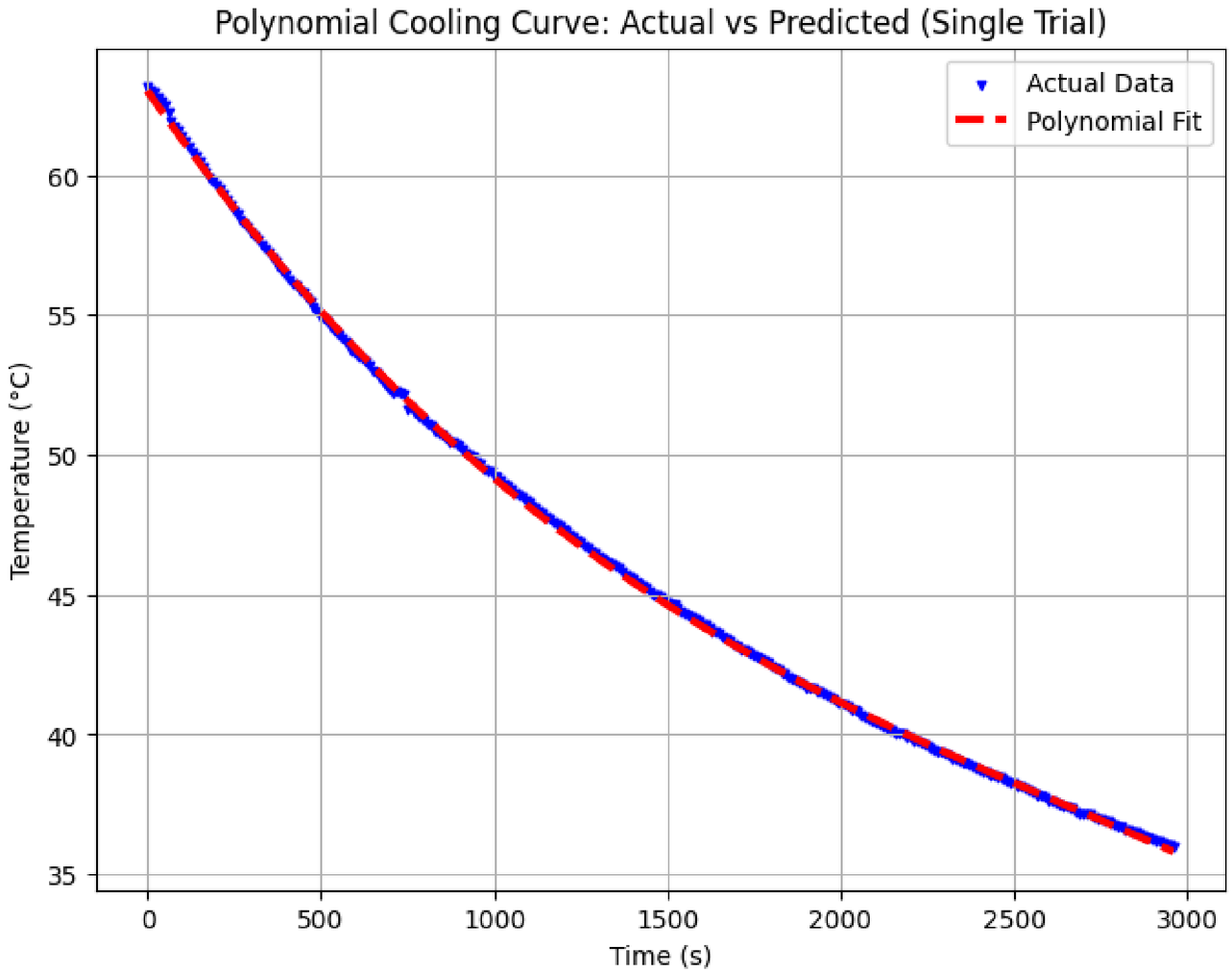
Results:

We got a 3rd degree polynomial fit for our data using sklearn

Performance Metrics:

```
Mean Squared Error: 0.01
R-squared: 1.00
```

This shows that curve fits data quite perfectly



Results:

Predictions:

The model is capable to predict the time the beverage will take to reach a suitable drinking temperature based on the initial temperature.

```
Predicted time to cool from 60°C to 40°C: 36.37 minutes
Predicted time to cool from 65°C to 40°C: 43.55 minutes
Predicted time to cool from 70°C to 40°C: 50.20 minutes
```

Limitations of Methodology

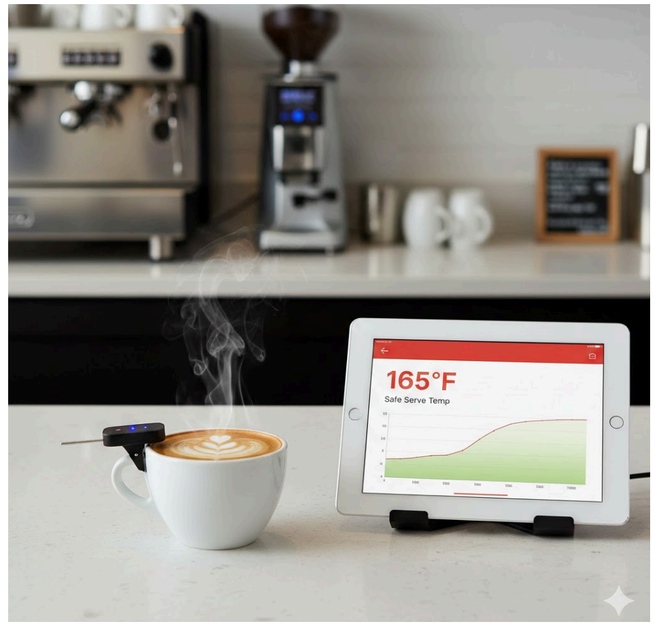
- Temperature readings can be affected by room conditions and airflow.
- Only one sensor was used, limiting accuracy.
- Polynomial regression used data from a single trial, not capturing all variations.
- Factors like cup material and beverage type were not varied.

Real World Implications – IoT

Smart Beverage Temperature Monitoring:

Application

- Monitoring temperature of hot beverages inservice environments
- Supporting safe and timely beverage serving



Benefits

- Minimizes risk of burns from hot liquids
- Enhances user comfort and satisfaction

Example

- Intelligent coffee machines indicating drink-ready time

Real World Implications – IoT

Health, Safety and Smart Home Integration:

Application

- Safe beverage consumption for sensitive users
- Integration with smart domestic systems

Benefits

- Alerts for safe drinking temperature
- Reduction oforal burn risks

Example

- Smart mugs providing temperature notifications.



Conclusion

- Successfully monitored hot beverage cooling using an Arduino-based system.
- Real-time temperature data was collected with a DS18B20 sensor and logged via Python.
- Polynomial regression accurately modeled the cooling curve.
- The model predicted the time to reach a safe drinking temperature (40°C).
- Demonstrates practical use of IoT and data analytics for beverage safety and quality control.

Thank You

We are happy to answer any questions?

Presented by:

- Aisha Khalid
- Khadim Hussain